

AEQUITAS NODE-BETREIBER-LEITFADEN

v2.0 · September 2026 · aequitas.digital

Einen Validator auf dem eigenen Server betreiben · Docker Compose · etwa 15 Minuten, davon 10 Bauen

Bevor du anfängst — Was du brauchst

1. Du bist ein registrierter Mensch: Aequitas-App installieren, die biometrische Registrierung abschließen, Wallet-Adresse notieren. Das Netz lehnt einen Validator ab, dessen Wallet kein registrierter Mensch ist — ein Mensch, ein Validator. Gemietete Server kaufen keine zusätzliche Stimme.
2. Ein Server (VPS) mit öffentlicher IPv4-Adresse: Ubuntu 22.04 oder 24.04, mindestens 4 vCPU, 8 GB RAM, 60 GB SSD (die Datenbank hat heute etwa 18 GB und wächst). Die beiden Gründerknoten laufen mit 6 vCPU / 12 GB / 100 GB. Die Ports 8080 (API) und 4001 (P2P) müssen aus dem Internet erreichbar sein.
3. Docker mit Compose-Plugin und git. Unter Ubuntu installiert `curl -fsSL https://get.docker.com | sh` beides.
4. Etwa 15 Minuten. Zehn davon ist der erste Build (der Knoten wird auf deinem Server aus dem Quelltext gebaut). Spätere Updates dauern genauso lang.

Schritt 1 — Konfiguration (.env)

Sicherheitshinweis: RELAYER_PRIVATE_KEY und NODE_KEY sind Geheimnisse. Wer sie hat, IST dein Knoten. Nie in Chat, E-Mail oder ein Ticket kopieren. Die .env-Datei bleibt auf deinem Server.

Variable	Pflicht?	Was eintragen
POSTGRES_PASSWORD	JA	Ein langes zufälliges Passwort für die lokale Datenbank. Nur die beiden Container auf deinem Server benutzen es.
SELF_URL	JA	Wie andere Knoten DEINEN erreichen: <code>http://DEINE-ÖFFENTLICHE-IP:8080</code> (oder <code>https://deine-domain</code> , wenn ein Proxy davor steht). Ohne SELF_URL folgt der Knoten der Kette nur als Beobachter und meldet sich nie als Validator an.
NODE_OPERATOR_WALLET	JA	Deine eigene Wallet-Adresse — MUSS ein registrierter Mensch sein (App-Registrierung abgeschlossen). Das bindet den Validator an eine Person. Empfängt die Validator-Belohnungen.
RELAYER_PRIVATE_KEY	Empfohlen	Der Schlüssel, der deine Blöcke signiert (0x..., 66 Zeichen). Am einfachsten: der private Schlüssel von NODE_OPERATOR_WALLET — dann ist keine Bindung nötig. Leer gelassen erzeugt der Knoten beim ersten Start einen und druckt ihn EINMAL („SAVE THIS AS ...“); in .env eintragen und neu starten, sonst wechselt deine Identität bei jedem Neustart.
NODE_OPERATOR_BINDING_SIGNATURE	Nur bei getrennten Schlüsseln	Wenn RELAYER_PRIVATE_KEY nicht der Schlüssel von NODE_OPERATOR_WALLET ist: die auf <code>aequitas.digital/node-binding</code> gezeigte Nachricht mit dem Wallet signieren und die Signatur hier eintragen.
NODE_KEY	Empfohlen	P2P-Identität. Wird beim ersten Start erzeugt und gedruckt, wenn leer — in .env sichern, aus demselben Grund wie oben.

Variable	Pflicht?	Was eintragen
PRIMARY_NODE_URLS	Voreingestellt t	Die Knoten, bei denen sich deiner anmeldet und von denen er beim ersten Start den Zustands-Snapshot holt. Voreingestellt auf die beiden Gründerknoten; so lassen.
GOMEMLIMIT	Voreingestellt t	Speicherdeckel des Knotens, voreingestellt 5GiB (passt zu 12 GB RAM). Bei 8 GB RAM 3GiB eintragen.
POSTGRES_SHARED_BUFFERS	Voreingestellt t	Postgres-Cache, voreingestellt 1GB. Ein Viertel des RAM ist ein guter Wert.

Schritt 2 — Knoten starten (Docker Compose)

Alles, was die beiden Gründerknoten tun, als eine Datei. Der erste Befehl baut den Knoten aus dem Quelltext (etwa 10 Minuten), startet Postgres und den Knoten und startet beide nach Absturz oder Reboot von allein neu.

- 1 Auf deinem Server: `git clone https://github.com/hanoi96international-gif/Aequitas.git`
- 2 `cd Aequitas/deploy/validator` und `cp .env.example .env`
- 3 `.env` bearbeiten (z. B. mit `nano .env`): `POSTGRES_PASSWORD`, `SELF_URL` und `NODE_OPERATOR_WALLET` ausfüllen — siehe Tabelle oben
- 4 `docker compose up -d --build` — baut und startet. Beim ersten Mal etwa 10 Minuten
- 5 Log ansehen: `docker compose logs -f node`. Beim ersten Start holt sich der Knoten den Netzzustand (Snapshot) von einem Gründerknoten, prüft dessen Signatur und zieht dann die Blöcke seitdem nach: [\[BOOTSTRAP\] Fresh node — importing state from ...](#), danach [\[HTTP-SYNC\] Added ... new blocks](#)
- 6 Stand im Log `SAVE THIS AS NODE_KEY` oder `SET THIS AS RELAYER_PRIVATE_KEY`: diese Werte jetzt in `.env` eintragen und `docker compose up -d` erneut ausführen — sonst bekommt dein Knoten bei jedem Neustart eine neue Identität
- 7 Sobald er aufgeholt hat, erscheinen `[Block #...]`-Zeilen — das sind Blöcke, die DEIN Knoten produziert. Bis dahin produziert er absichtlich nichts (Frischer Knoten: ... produziert nichts, bis er aufgeholt hat) — ein Knoten, der die Kette nie gesehen hat, darf keine erfinden

```
# deploy/validator/.env — die drei Pflichtwerte
POSTGRES_PASSWORD = ein-langes-zufaelliges-passwort
SELF_URL          = http://DEINE-OEFFENTLICHE-IP:8080
NODE_OPERATOR_WALLET = 0xDEIN_MENSCH_WALLET
# empfohlen: der Schluesel, der deine Bloecke signiert (oder beim ersten Start leer lassen)
RELAYER_PRIVATE_KEY = 0xDEIN_PRIVATER_SCHLUESSEL
# voreingestellt, so lassen
PRIMARY_NODE_URLS  = http://173.249.37.118:8080,http://194.163.188.71:8080
```

Schritt 2b — Aktualisieren, neu starten, stoppen

Der Knoten hält seinen Zustand in zwei Docker-Volumes (Datenbank und Überweisungs-Log). Ein Update baut das Image aus dem neuesten Code neu; der Zustand bleibt. Starte nie alle Validatoren des Netzes gleichzeitig neu — einen nach dem anderen.

```
# Auf den neuesten Code aktualisieren (etwa 10 Minuten)
cd Aequitas && git pull && cd deploy/validator && docker compose up -d --build

# Nur den Knoten neu starten
docker compose restart node
```

```
# Alles stoppen (Zustand bleibt erhalten)
docker compose down

# Ressourcen beobachten
docker stats
```

Schritt 3 — Prüfen, ob dein Knoten läuft

Vergleiche deinen Knoten mit dem Netz. DEINE-ÖFFENTLICHE-IP durch deine Serveradresse ersetzen.

```
curl -s http://DEINE-OEFFENTLICHE-IP:8080/api/status | grep -oE '"height":[0-9]+'
curl -s https://aequitas.digital/api/status | grep -oE '"height":[0-9]+'
→ Beide Zahlen müssen bis auf ein paar Blöcke gleich sein.

http://DEINE-OEFFENTLICHE-IP:8080/ → dein eigener Explorer
http://DEINE-OEFFENTLICHE-IP:8080/api/health/combined → alles, was der Knoten über sich misst
```

Direkt nach dem ersten Start liegt die Höhe weit unter dem Netz, solange der Snapshot importiert und die jüngsten Blöcke nachgezogen werden. Bleibt sie länger als 15 Minuten weit zurück: Log auf [BOOTSTRAP]-Fehler prüfen und ob die Ports 8080 und 4001 offen sind.

Schritt 3b — Wallet an den Knotenschlüssel binden (nur bei getrennten Schlüsseln)

Wenn RELAYER_PRIVATE_KEY nicht der private Schlüssel von NODE_OPERATOR_WALLET ist, beweise einmal, dass beide dir gehören: die Seite unten öffnen, die gezeigte Nachricht mit deinem Wallet signieren und die Signatur als NODE_OPERATOR_BINDING_SIGNATURE in .env eintragen.

<https://aequitas.digital/node-binding>

Mit dem einfachen Ein-Schlüssel-Aufbau (RELAYER_PRIVATE_KEY = Schlüssel deines Wallets) entfällt dieser Schritt.

Schritt 4 — MetaMask mit deinem Knoten verbinden (optional)

In MetaMask: Netzwerk-Auswahl → Netzwerk hinzufügen → Manuell hinzufügen, dann eintragen:

Netzwerkname	Aequitas Chain
RPC-URL	http://DEINE-OEFFENTLICHE-IP:8080/rpc
Chain-ID	1926
Währungssymbol	AEQ
Dezimalstellen	18
Block-Explorer	https://aequitas.digital

Schritt 5 — Validator-Belohnungen

Der Validatoren-Pool erhält 40 % aller Swap-Gebühren. Täglich um 20:00 Uhr Berliner Zeit wird er zu gleichen Teilen an alle Validatoren verteilt, die registrierte Menschen betreiben — gewichtet nur nach den Minuten, die der Knoten an dem Tag online war, nicht nach Blöcken; teure Hardware bringt also nicht mehr. Transfergebühren, Liegegeld und Deckel-Überschuss gehen vollständig ins Grundeinkommen, nicht an Validatoren. Außer den Knoten laufen zu lassen, ist nichts zu tun.

- 1 **NODE_OPERATOR_WALLET** muss ein registrierter Mensch sein — sonst lehnt das Netz die Anmeldung ab (Log: `NODE_OPERATOR_WALLET is not a registered human`).
- 2 Im Log bestätigen: `[PEERS] Auto-authorized validator ... (wallet: 0x...)` auf einem Gründerknoten und [Block #...]-Zeilen auf deinem.
- 3 Ein Knoten, der nicht läuft, verdient in dieser Zeit nichts: Der Anteil zählt nur die Minuten, in denen Ihr Knoten online war. Neustarts sind harmlos, der Knoten holt von allein auf.
- 4 Was ein Validator ohne zusätzliche Software NICHT tut: neue Menschen registrieren (diese Endpunkte antworten 503). Überweisungen, Blöcke und Belohnungen funktionieren ohne das.

Fehlerbehebung

Symptom	Wahrscheinliche Ursache	Lösung
NODE_OPERATOR_WALLET is not a registered human	Das Wallet hat die App-Registrierung nicht abgeschlossen	Erst in der App registrieren, dann den Knoten neu starten.
operator_binding_signature missing or invalid	Signierschlüssel und Wallet verschieden, keine Bindung	Schritt 3b: auf <code>/node-binding</code> signieren und <code>NODE_OPERATOR_BINDING_SIGNATURE</code> setzen.
Frischer Knoten: ... produziert nichts, bis er aufgeholt hat	Normal beim Aufholen	Warten. Die Produktion beginnt, sobald der Knoten saubere Sync-Zyklen mit den Gründerknoten hat.
SELF_URL not set — ... Beobachter	SELF_URL fehlt in <code>.env</code>	SELF_URL auf <code>http://DEINE-OEFFENTLICHE-IP:8080</code> setzen und <code>docker compose up -d</code> ausführen.
Höhe bleibt weit unter dem Netz	Snapshot-Import gescheitert oder Ports zu	Log auf [BOOTSTRAP]-Zeilen prüfen; TCP 8080 und 4001 eingehend in Firewall / Cloud-Sicherheitsgruppe öffnen.
Knoten startet in Schleife neu, „OOMKilled“	Zu wenig RAM	GOMEMLIMIT in <code>.env</code> senken (3GiB bei 8 GB RAM) oder dem Server mehr Speicher geben.
docker compose: Build schlägt fehl	Kein ausgehendes Internet beim Bauen	Der Build lädt Go-Module; DNS und ausgehendes HTTPS auf dem Server prüfen.
Log zeigt [AUTO-HEAL] ... resync	Der Knoten ist zurückgefallen, an einem Block hängen geblieben oder sein Kontostand weicht von den Gründerknoten ab	Nichts zu tun: der Knoten holt den signierten Snapshot von selbst neu und produziert danach weiter. Nur wenn sich das stündlich wiederholt, Platte und RAM prüfen.